

Version 1.2

August 2026



MACHINE LEARNING

MODULE 3 - IMAGE CLASSIFICATION

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE OBJECTIVES.....	3
PREPARATION.....	3
MAIN TOPICS.....	4
1. IMAGE.....	4
2. IMAGE CLASSIFICATION.....	4
3. CNN.....	5
1. Convolutional Layer.....	5
2. Pooling Layer.....	6
3. Flatten Layer.....	8
4. Fully Connected Layer.....	9
4. Data Preprocessing.....	10
1. Resizing.....	10
2. Normalization.....	11
3. Data Type Conversion.....	11
Building a CNN Model Using TensorFlow and Keras.....	12
1. Load Dataset.....	12
2. Importing Library.....	12
3. Preprocessing Data.....	13
4. Build CNN Model.....	15
5. Compile Model.....	17
6. Training Model.....	18
7. Evaluate Model.....	19
8. Image Classification Function.....	20
CODELAB (10%).....	22
LAB ASSIGNMENT (90%).....	26



INTRODUCTION

OBJECTIVES

1. Students understand the basic concepts and types of images.
2. Students understand image classification and Convolutional Neural Networks (CNN).
3. Students understand the basic CNN architecture, including the Convolutional Layer, Pooling Layer, Flatten Layer, and Fully Connected Layer.
4. Students understand the importance of image data preprocessing and common techniques used.
5. Students can build and train a CNN model using TensorFlow and Keras for image classification.
6. Students can evaluate trained CNN models.

MODULE OBJECTIVES

1. Students can improve image data quality to make it cleaner and more relevant for the model.
2. Students can standardize data so all images have consistent formats and sizes.
3. Students can support the model training process by helping it learn important patterns in the data.
4. Students can improve model performance to be more accurate and robust (resistant to variations).

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
 رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop/PC
3. Google Colaboratory.
4. TensorFlow >= 2.10, Keras, matplotlib.pyplot, numpy, os.
5. Source Code : [🔗 MODULE 3 MACHINE LEARNING - IMAGE CLASSIFICATION](#)



MAIN TOPICS

1. IMAGE

An image, or more commonly called a picture, is a visual representation of an object or scene that can be a drawing, photograph, or graphic. Examples include photos of cats, landscapes, or hospital X-rays. So, how do computers see images? Computers cannot see images the way human eyes do. So, computers store images as collections of pixels. The more pixels an image has, the clearer and sharper it will be.

Images are divided into two types: color images and grayscale images. Color images have pixels that store red, green, and blue (RGB) color information. Combining these three colors produces millions of other colors. Meanwhile, grayscale images, commonly called grayscale, have only black, white, and shades of gray. Each pixel stores only a brightness level, from dark to light. These images are often used in data analysis because they are simpler for computers to process.

2. IMAGE CLASSIFICATION

Image classification is one of the main tasks in computer vision. Its goal is to teach computers to recognize and group objects in an image. For example, when you see a photo of a dog, your brain automatically recognizes it as a dog. In this case, the image is processed and labeled so the computer can recognize and classify objects in it.

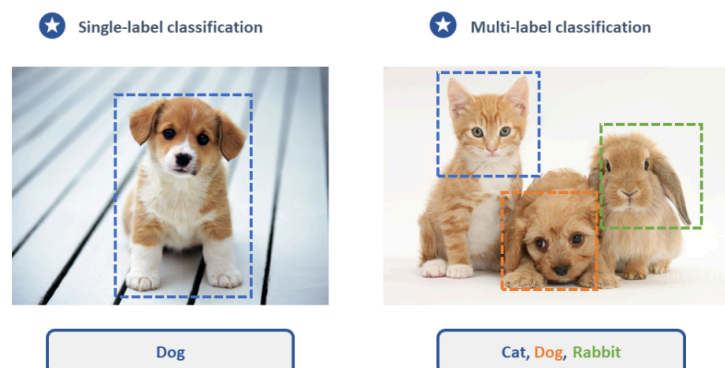


Figure 1. Comparison between Single-Label and Multi-Label Image Classification



Single-label classification classifies an object into only one category. For example, when you classify images of cats vs. dogs. The model must choose one of two labels, namely “cat” or “dog”. One photo cannot be labeled “cat” and “dog” at the same time. In essence, single-label classification has only one correct output. Meanwhile, multi-label classification allows an object to have more than one category or label at the same time.

3. CNN

A Convolutional Neural Network (CNN) is a type of neural network designed to process visual data, such as images. CNNs have become one of the most successful methods in computer vision because they can recognize visual patterns automatically, without us having to tell them exactly what to look for. Simply put, CNNs are inspired by how the human brain works, so this architecture works like our eyes and brain when looking at an object. The human eye does not process the entire image at once; instead, it recognizes an object by first identifying small features—such as lines, corners, or colors—then combining these features into more complex shapes.

A CNN has several main layers, each with specific functions that process data from the previous layer.

1. Convolutional Layer

The convolutional layer is the core layer of a CNN that is useful for extracting features from images. It uses a filter/kernel, a small matrix of numbers. This filter/kernel finds specific patterns in the image, such as edges, corners, or textures. This layer produces a feature map.

In CNNs, image data is generally represented in three dimensions: Height (H), Width (W), and Number of Channels (C). For example, an RGB image (3 color channels) measuring 5x5 pixels will have an input dimension of (5x5x3), where 3 is the number of color channels. To process this image, a CNN uses a filter/kernel with three dimensions: height (K), width (K), and number of channels (C). The number of channels must match the input image. For example, for an RGB image (5 x 5 x 3), a commonly used kernel size is (3 x 3 x 3).

In general, the process of the convolutional layer can be formulated as follows:

$$feature_surface_{out} = f \left(\sum_{i=1}^3 M_i * W_i + B \right)$$



In this formula, M_i represents the size of the input feature, W_i is the weight matrix of the convolution kernel, $f(\cdot)$ is a non-linear activation function (e.g., ReLU), and $\text{feature_surfaceout}$ is the output feature. If we use the Keras library to create a convolutional layer, the code is as follows:

```
Input(shape=(H, W, 3)),
Conv2D(32, (3, 3), activation='relu'),
```

`Input(shape=(H, W, 3))` is the definition of our input image. The number 3 here refers to the number of color channels (C) in the image, usually RGB (Red, Green, Blue). This corresponds to the components M_i and C in the formula, where M_i is the feature data on each input channel (e.g., Red, Green, and Blue channels), and C is the total number of channels (in this case, 3), which becomes a limit for the sum operation (Σ) in the formula.

Meanwhile, in the second line, `Conv2D` performs two-dimensional convolution, where 32 is the number of filters (kernels) used, meaning the convolution produces 32 feature maps. Furthermore, the parameter (3,3) is the size of each kernel. In short, this line of code implements the entire mathematical process described by the formula. The code `Conv2D(32, (3, 3), activation='relu')` will automatically perform the multiplication between the input (M_i) and the kernel (W_i), sum the results (B), and activate it with a function (f), then repeat it 32 times to produce 32 Feature Maps. For an illustration, observe the following diagram:

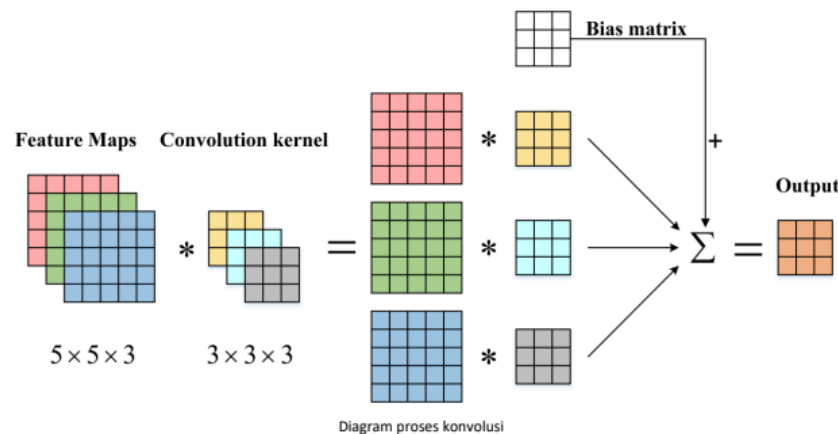


Figure 2. Multi-channel convolution operation workflow with bias addition

2. Pooling Layer

The pooling layer will receive feature maps from the previous layer. The pooling layer simplifies the existing information (downsampling). This layer reduces the size of the feature map by taking the most important values from the feature map so that only the essence of the



features is retained; thus, the pooling layer will make the model more efficient in terms of computation and help it become less sensitive to small shifts in the position of objects in the image.

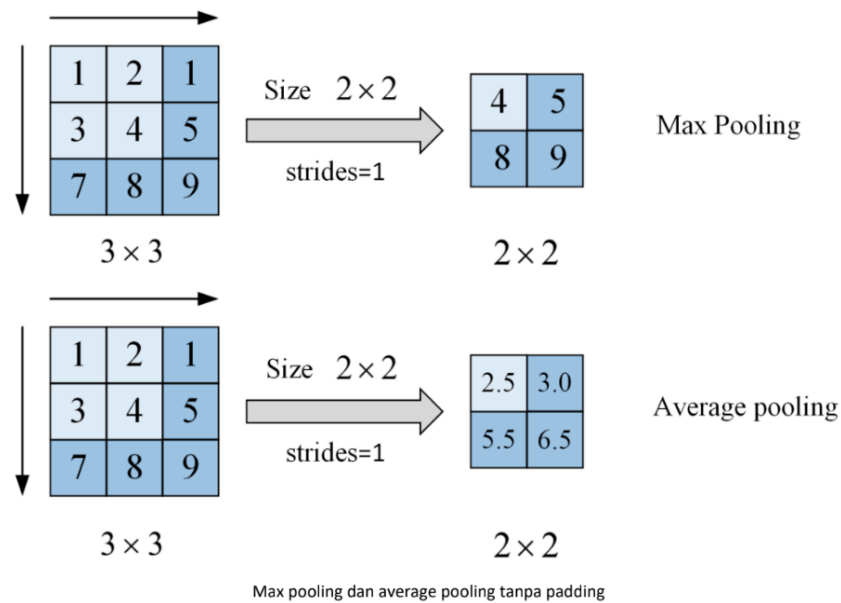


Figure 3. Comparison between Max Pooling and Average Pooling Operations without Padding

Common pooling layers include max pooling and average pooling. In short, max pooling takes the maximum value from each small window that slides across the feature maps. This makes the model less sensitive to small shifts (translations) in object position. Meanwhile, average pooling calculates the average value in each small window as it slides across the feature maps. Unlike max pooling, which focuses on the most dominant features, average pooling provides a general overview or average of the features in each area. This method suits cases where evenly distributed details matter more than prominent single features. However, these two types of pooling are not necessarily the best; other types exist, but these are the most commonly used in CNNs.



3. Flatten Layer

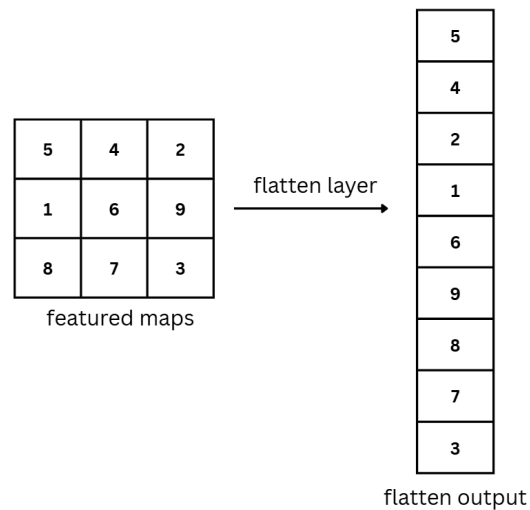


Figure 4. Flattening process of a 2D feature map into a 1D vector

After passing through the convolutional and pooling layers, which build more complex features, the flatten layer transforms the summarized information into a single long data vector. The flatten layer prepares the data for the final stage as a vector. This data vector, which now contains a numerical representation of all the important extracted features, is then fed into the next layer.

For example, if a pooling layer produces a feature map of size 3x3x5, it has 45 elements. When this data passes through the flatten layer, it is converted into a one-dimensional vector of length 45. So, the Flatten Layer is the data “preparation” stage. It takes all the feature information found by the CNN and presents it in a format the classification layer can understand, allowing the model to make accurate final predictions.



4. Fully Connected Layer

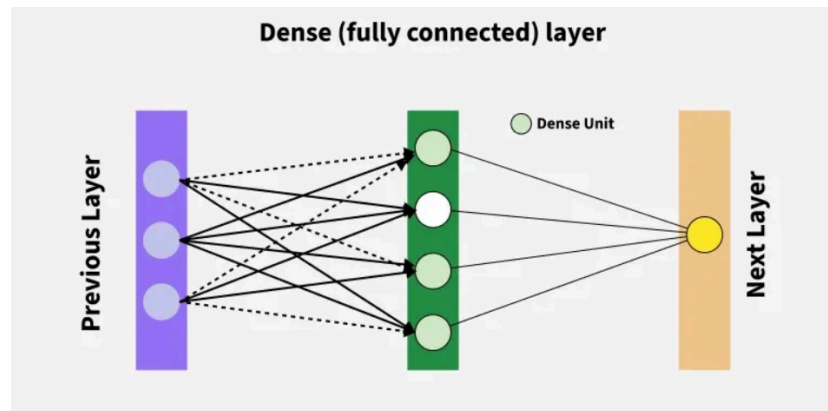


Figure 5. Structure and connectivity of a dense (fully connected) layer

The fully connected layer, commonly called the dense layer, analyzes relationships among all existing features to make a final decision. This layer accepts only one-dimensional input, so CNN models need a flatten layer. Analogously, if the flatten layer is a bridge, then the fully connected layer is the brain that makes classification decisions. By combining the extracted features, the fully connected layer uses trained weights and biases to identify complex patterns and make classification decisions. The output of this layer, especially at the end of the CNN architecture, is often probabilities for each class, indicating how confident the model is about each possible category, such as "cat" or "dog".

For clarity, see the CNN diagram below.

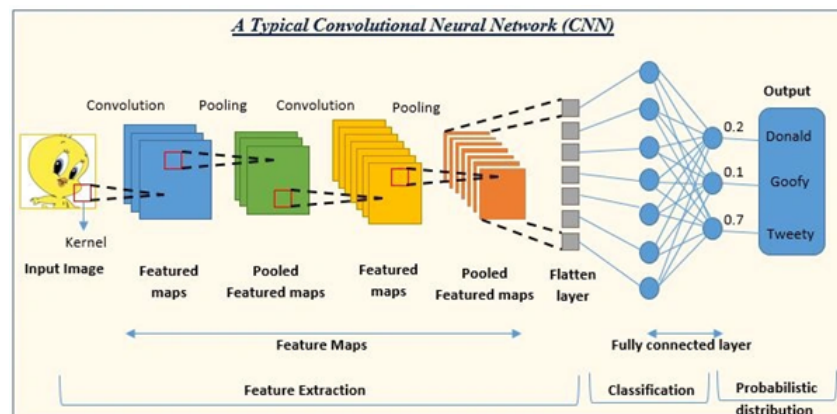


Figure 6. End-to-end CNN workflow showing feature extraction and classification stages



In the diagram above, the input image given is a “tweety” image. The image is then processed through the convolutional layer to extract features. In this layer, a small matrix called a kernel/filter will slide across the entire image. Each time the kernel/filter slides, it will perform calculations that produce a single value. These values are collected into a feature map.

Next, the feature map goes to the pooling layer to summarize the data. The result is usually called a pooled feature map. It is smaller, but it retains the essential features.

In the diagram above, after a pooling layer, a convolutional layer and another pooling layer are applied again. These two layers are usually repeated several times to reprocess the previous layers, combining simple features into more complex ones at each stage.

After feature extraction, the results enter the classification stage. Here, the Flatten layer transforms the final feature maps into a single long row of data. This data is then fed into the Fully Connected Layer, which combines all extracted feature information. This layer will learn patterns from these features to make a final decision. The Output Layer represents this decision as probabilities. Each neuron in this layer represents a category, and its probability value indicates how confident the model is that the image belongs to that category.

For example, after going through the entire process, the model predicts that the image is “Tweety” with 70% probability, which is the highest confidence. This step-by-step flow allows a CNN to “learn” and “see” images similarly to humans, from small features to overall object recognition.

To understand it more clearly, watch the following video: [LINK VIDEO CNN](#).

4. Data Preprocessing

In data mining courses, you must be familiar with the term data preprocessing. Not only text data, but image data can also be pre-processed. The goals are to:

- Improve the quality of image data to be cleaner and more relevant for the model
- Standardize data so that all images have consistent formats and sizes
- Help the model training process learn important patterns in the data
- Improve model performance to be more accurate and robust (resistant to variations)

Here are some common preprocessing techniques.

1. Resizing

CNN models usually expect a fixed input size. If our dataset has images of various sizes, we need to resize them to the same dimensions.



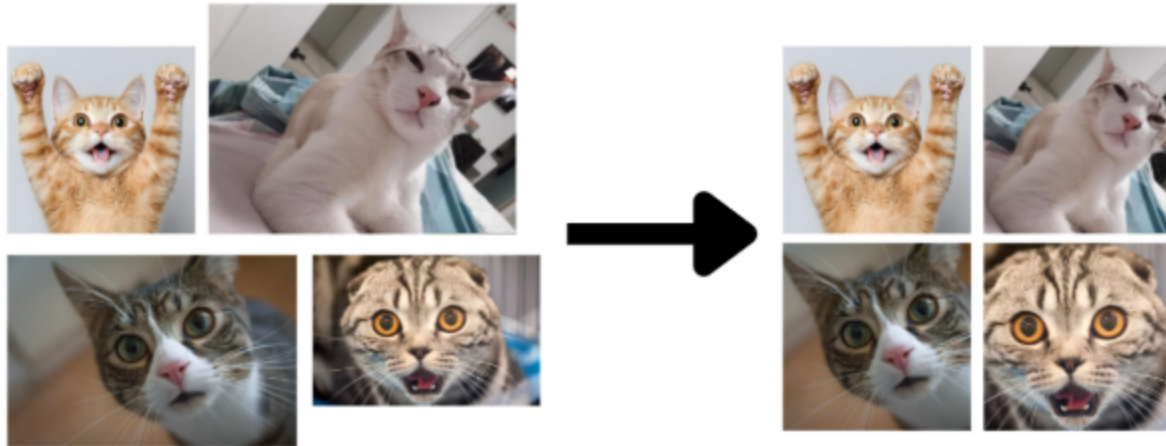


Figure 7. Comparison of images before and after size normalization

For example, we might resize all images to 224x224 pixels or 128x128 pixels, depending on your needs. The implementation in code is as follows:

```
resized_image = tf.image.resize(image, (128, 128))
```

2. Normalization

Normalization transforms pixel values into a standard range, usually between 0 and 1 or -1 and 1. This helps speed up model training and prevents a single pixel with a very high value from dominating the learning process.

```
normalized_image = resized_image / 255.0
```

3. Data Type Conversion

Before entering the model, image data needs to have the correct numerical data type. Generally, normalized image data (range 0-1) should use floating-point to ensure accurate mathematical calculations.

```
final_image = tf.cast(normalized_image, dtype=tf.float32)
```



Building a CNN Model Using TensorFlow and Keras

For the interactive example, please click this link: [COLAB MODULE 3](#)

If the data is image data, the most commonly used and recommended framework is TensorFlow with the Keras library. This is because TensorFlow serves as the "engine" that executes the commands we provide through Keras, such as matrix multiplication, convolution operations, and weight optimization. Meanwhile, Keras hides the mathematical and computational complexity behind the scenes, letting us focus on the model architecture, such as defining layers (e.g., Conv2D, MaxPooling2D, Flatten, and Dense). If you are using it outside of Google Colab, make sure TensorFlow is installed; if not, use the command below:

```
! pip install tensorflow
```

1. Load Dataset

Next, make sure you have the dataset installed. You can install it directly from Google Colab or manually. Access [this dataset](#) so you can directly understand the material below. (If not, do not run the code below so that the output is not lost).

2. Importing Library

If the data is image data, the most commonly used and recommended framework is TensorFlow with the Keras library. This is because TensorFlow serves as the "engine" that executes the commands we provide through Keras, such as matrix multiplication, convolution operations, and weight optimization. Meanwhile, Keras hides the mathematical and computational complexity behind the scenes, letting us focus on the model architecture, such as defining layers (e.g., Conv2D, MaxPooling2D, Flatten, and Dense).

If you are using it outside of Google Colab, make sure TensorFlow is installed; if not, use the command below:

```
! pip install tensorflow
```



```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

- ImageDataGenerator performs image data augmentation and loads image data from a directory into the model in the correct format.
- Sequential is the simplest and most commonly used API model. So, Sequential is useful for building a neural network model by stacking several layers one by one. Sequential acts as a container for building the model.

If so, create a variable to store your dataset path.

```
base_dir = 'animals'
```

3. Preprocessing Data

Next, initialize ImageDataGenerator to preprocess images and create data generators for training and validation.

```
datagen = ImageDataGenerator(rescale=1./255)

train_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary')

validation_generator = datagen.flow_from_directory(
    base_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary')
```



Explanation of arguments for train_generator and validation_generator:

- `base_dir`: The first argument; a variable pointing to the main directory where the image dataset is located ('animals'). `flow_from_directory` will look for subdirectories within `base_dir`. In this dataset, there are 'cat' and 'dog' subdirectories. This method will automatically consider the subdirectory names as class names.
- `target_size=(150, 150)`: This argument specifies the size (height, width) to which all images will be resized. Although the original images may have different resolutions, the generator will resize them to 150x150 pixels before feeding them into the model. This matters because CNN models require consistent image input sizes.
- `batch_size=32`: This sets the batch size of data the generator produces. During training, the model does not process all images at once; instead, it processes them in small groups called batches. A batch size of 32 means that each time the model requests data from the generator, the generator will provide 32 images and their corresponding labels.
- `class_mode='binary'`: This argument specifies the type of labels to be generated. Since we are doing binary classification (pizza vs not pizza, i.e., 2 classes), we use 'binary'. This will make the generator produce binary labels (0 or 1) for each image, corresponding to its class. If you have more than two classes, you would use 'categorical'

So, what's the difference between the two generators if they have the same arguments? Because we only have two dataset subfolders, namely 'cat' and 'dog', without separating them into train, test, and validation sets. Thus, the arguments used are the same. If two generators have the same arguments, they access the same data source. Both will read images from the same directory (`base_dir`), with the same target size, batch size, and class mode. The main difference is their purpose during model training.

- `train_generator` is used in `model.fit()` to provide batches of data the model uses to learn and update its weights.
- `validation_generator` is also used in `model.fit()`, but it provides batches of data to monitor model performance during training without updating the model's weights.



This validation result helps identify if the model is starting to overfit—for example, performing well on training data but declining on validation data. However, if both generators read from the same directory, the validation process runs on the same data as training. This can give a misleading picture of validation performance and mask potential overfitting problems. In practice, validation_generator should read from a separate validation dataset that the model has never seen during training.

4. Build CNN Model

To build a CNN model, we need a conceptual understanding of how the model will process images, as explained in point 3: CNN, about the layers needed to identify visual patterns that distinguish between the two images. This process largely happens through a feature-extraction stage that combines convolution and pooling layers. Before building the model, let's imagine what layers our model needs to classify pizza and non-pizza images. First, we need a convolution layer to extract basic features such as lines, edges, or textures from the image. Next, we need a pooling layer to reduce the representation size while preserving important information. This combination of convolution and pooling is usually repeated several times so the model can recognize increasingly complex patterns, such as the characteristic shape of a pizza or the features of a non-pizza image.

Next, the feature-extraction results pass through a flatten layer to convert them into a one-dimensional vector, then go to a dense layer so the model can make decisions. Finally, we add an output layer with sigmoid activation to produce the probability that the given image is pizza. If written in code, it will look like the following:

```
model = Sequential([
    Conv2D(16, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.summary()
```

You can modify the CNN layers to fit your dataset.



- `Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3))`: This is the first layer in the model, which is a 2D convolutional layer (`Conv2D`).
- `32`: The number of convolutional filters (kernels) in this layer. Each filter will learn to detect different features in the image.
- `(3, 3)`: The size of each convolutional filter (height x width). Here, the filter is 3x3 pixels.
- `activation='relu'`: The activation function used after the convolution operation. ReLU (Rectified Linear Unit) is a commonly used and effective activation function.
- `input_shape=(150, 150, 3)`: This only needs to be specified in the first layer of a Sequential model. It defines the expected input shape: 150 pixels high, 150 pixels wide (adjust to your image data size), and 3 color channels (for RGB images).
- `MaxPooling2D((2, 2))`: This is a 2D max pooling layer. `(2, 2)`: The size of the pooling window. This layer takes the maximum pixel value from each 2x2 pixel block in the previous layer's output. This reduces the output's spatial dimensions (height and width), lowers the number of parameters, and makes the model more robust to small variations in feature position.
- `Flatten()`: This layer transforms the output of the previous layer (which is 3D: height, width, channels) into a 1D vector (one-dimensional). This is necessary because the subsequent Dense (fully connected) layer expects 1D input.
- `Dense(64, activation='relu')`: This is the first Dense (fully connected) layer. `64`: The number of units (neurons) in this layer. Each neuron is connected to all neurons in the previous layer.
- `Dense(1, activation='sigmoid')`: This is the last Dense layer (output layer). Here, `1` is the number of units in the output layer. Since this is a binary classification problem (pizza vs. not pizza), we only need one output unit.
- `activation='sigmoid'`: The sigmoid function produces an output between 0 and 1, which can be interpreted as the probability of the positive class (e.g., the probability that the image is a dog).
- `model.summary()`: This method prints a summary of the model, including the name of each layer, its output shape, and the number of trainable parameters in each layer. This helps you understand the model structure and how many parameters it needs to learn. Thus, from the code above, we can see a model overview like this:



Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 16)	448
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 32)	1,327,136
dense_1 (Dense)	(None, 1)	33

Total params: 1,332,257 (5.08 MB)
 Trainable params: 1,332,257 (5.08 MB)
 Non-trainable params: 0 (0.00 B)

So, there is no rigid rule that determines the exact number of layers to use. Instead, the number of layers in a CNN model highly depends on several factors:

- Problem complexity: Classifying pizza and non-pizza is a relatively simple case compared to classifying thousands of different objects. For more complex problems, you usually need more layers to extract finer features and differentiate between classes.
- Image data size and quality: For example, if the image data used is small, low-resolution, or noisy, a model with too many layers will tend to overfit (the model memorizes the training data too much, and its performance will be poor on new data). Conversely, if the data we have is large and high-quality, more layers can help the model learn richer representations.
- Availability of Computational Resources: Models with more layers require more computational power and training time. If resources are limited, a simpler architecture with fewer layers might be more practical.

5. Compile Model

So that the model can be trained, we must compile (prepare/set up the model before training) it:

```
from tensorflow.keras.optimizers import Adam

model.compile(optimizer=Adam(),
              loss='binary_crossentropy',
              metrics=['accuracy'])
```



As explained in the previous module, we will use the Adam optimizer. Loss is a function the model uses during training to measure how well or poorly it performs on the training data. The goal is to minimize the loss value. Meanwhile, 'binary_crossentropy' is the specific loss function used here. Binary crossentropy is a very common and recommended loss function for binary classification problems (when you have two classes, such as pizza vs. non-pizza). This function measures the difference between the probability distribution predicted by the model (the output of the sigmoid layer, which is a probability between 0 and 1) and the actual probability distribution (binary labels 0 or 1). The smaller the binary_crossentropy value, the closer the model's predictions are to the actual labels. So, by setting loss='binary_crossentropy', we tell the model to use this function as a measure of "error" during training, and the optimization process (specified by optimizer=Adam()) will try to reduce this error.

6. Training Model

If the model is ready, we just need to train the model with the dataset we prepared earlier.

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

epochs = 7
history = model.fit(
    train_generator,
    epochs=epochs,
    validation_data=validation_generator,
    callbacks=[EarlyStopping(monitor='val_loss', patience=5), ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
    verbose=1, min_lr=1e-7)]
)
```

model.fit() runs training using the data we provided in the specified parameters, such as training data (train_generator), number of epochs, validation data (validation_generator), and callbacks. What are callbacks? Callbacks are processes in model training that allow certain functions or actions at specific points during training. Or, more clearly, a callback is a function or object that we give to model.fit() so that this callback runs automatically at certain moments during model training.

For example, we have 15 epochs to train the model to achieve its highest accuracy, but after training through the last epoch, the model's accuracy actually decreases. After visual analysis, we found that the model achieved its highest accuracy at epoch 7. But it's not certain that reducing epochs to the peak accuracy will yield the same accuracy, and retraining the



model would waste time. This is where callbacks are very useful. Here are some examples of commonly used callbacks:

- EarlyStopping: Automatically stops training if the model's performance doesn't improve, preventing overfitting.
- ModelCheckpoint: Saves the best weights when the model's performance peaks, even if training continues until the end.
- ReduceLROnPlateau: Useful for lowering the learning rate if the model's performance cannot improve. It's like this: when the model is stuck at a certain performance level, this function lowers the learning rate so it can find the optimal point again.
- There are many other types of callbacks. For a more complete explanation and their parameters, see the following documentation: [API CALLBACK KERAS](#).

7. Evaluate Model

```
test_dir = base_dir # Using the same directory for testing as the dataset doesn't have a separate test set
test_datagen = ImageDataGenerator(rescale=1./255)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary',
    shuffle=False
)

loss, accuracy = model.evaluate(test_generator)

print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

Found 1966 images belonging to 2 classes.
62/62 ————— 11s 185ms/step - accuracy: 0.9618 - loss: 0.1280
Test Loss: 0.1069
Test Accuracy: 0.9741
```

Then evaluate the model using test data so it can predict images beyond those provided in the training and validation data, as in the code above. Perform further evaluation as needed.



```
from sklearn.metrics import classification_report
import numpy as np

true_classes = test_generator.classes
predictions = model.predict(test_generator)
predicted_classes = np.round(predictions).flatten()

class_labels = list(test_generator.class_indices.keys())

report = classification_report(true_classes, predicted_classes, target_names=class_labels)
print(report)
```

	precision	recall	f1-score	support
not_pizza	0.98	0.97	0.97	983
pizza	0.97	0.98	0.97	983
accuracy			0.97	1966
macro avg	0.97	0.97	0.97	1966
weighted avg	0.97	0.97	0.97	1966

8. Image Classification Function

If the model has been evaluated using metrics, we will perform classification using the trained model. Use the following code:

```
import numpy as np
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

...
Tentukan parameter sesuai kebutuhan kalian,
target size disesuaikan dengan ukuran citra saat kalian melatih model
...
def classify_image(image_path, model, target_size=(150, 150)):
    img = image.load_img(image_path, target_size=target_size) #Memuat gambar dan mengubah ukuran sesuai dengan target size
    img_array = image.img_to_array(img) #Mengubah objek gambar menjadi array numpy agar sesuai dengan input yang dibutuhkan CNN
    img_array = np.expand_dims(img_array, axis=0) #Menambahkan dimensi baru di awal array karena model membutuhkan input dalam bentuk batch
    img_array /= 255.0 #Normalisasi nilai piksel pada gambar (sesuai dengan preprocessing yang dilakukan saat melatih model)

    prediction = model.predict(img_array)
    probability = prediction[0][0] #Membuat prediksi pada array citra yang sudah diproses

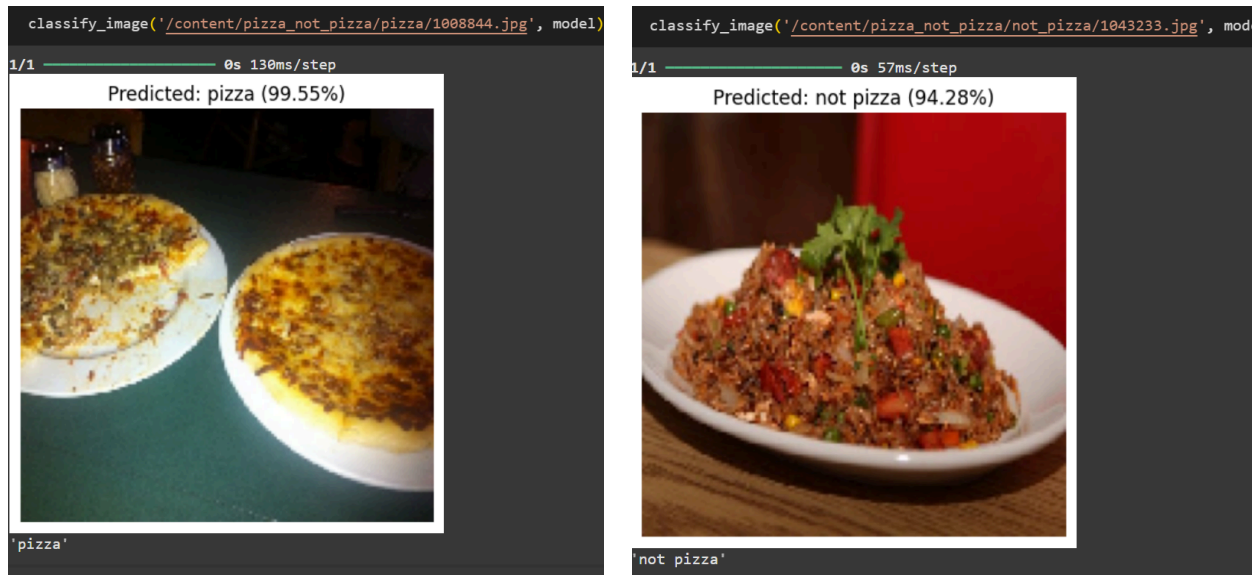
    #Mengambil nilai probabilitas dari hasil prediksi
    if probability > 0.5:
        predicted_class = 'pizza'
        confidence = probability * 100
    else:
        predicted_class = 'not pizza'
        confidence = (1 - probability) * 100

    plt.imshow(img)
    plt.title(f'Predicted: {predicted_class} ({confidence:.2f}%)')
    plt.axis('off')
    plt.show()

    return predicted_class
```



Thus, to use the function above, you can use the following code:



So, when the model has been trained, it doesn't mean "accuracy appears = finished". Instead, we can experiment by providing image inputs outside the training and validation data to test whether the model's performance matches the results during training and evaluation.



CODELAB (10%)

Download the following dataset: [CODELAB DATASET](#)

Complete and correct the code below, then train the model to achieve good performance.

1. Load the dataset. (10 Points)

```
Load Dataset

!unzip .... .zip
```

2. Preprocessing Data. (10 Points)

```
from tensorflow.keras.preprocessing.image import ...

dataset_path = '...'

datagen = ImageDataGenerator(rescale=..., validation_split=...)

train_generator = datagen.flow_from_directory(
    dataset_path,
    target_size=(...),
    batch_size=...,
    ...='binary',
    subset='training'
)

validation_generator = datagen.flow_from_directory(
    dataset_path,
    target_size=(...),
    batch_size=...,
    ...='binary',
    subset='validation'
)

test_generator = datagen.flow_from_directory(
    dataset_path,
    target_size=(...),
    batch_size=...,
    ...='binary',
    subset='validation',
    shuffle=...
)
```



3. Build CNN Model. (20 Points)

```
from tensorflow.keras.... import ...
from tensorflow.keras.... import ...

model = Sequential([
    ...
    ...
])

model.summary(fit())
```

4. Compile Model. (20 Points)

```
from tensorflow.keras.optimizers import ...

model.compile(optimizer=...,
              loss='...',
              metrics=['...'],
              callbacks=[...])
```

5. Training Model. (10 Points)

```
epoch = ...
history = model.compile(
    ...,
    epochs=...,
    validation_data=...,
    callbacks=[...]
    metrics=[...]
)
```

6. Evaluate Model. (10 Points)

```
loss, accuracy = model.evaluate(...)

print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")
```



7. Image Classification Function. (20 Points)

```
import ... as np
from tensorflow.keras.preprocessing import ...
import ... as plt

def classify_new_image(...):
    img = ...
    img_array = image...(img)
    ...
    ...
    ...

    prediction = model.predict(img_array)
    probability = prediction[2][2]
    if ... > 0.5:
        predicted_class = 'car'
        confidence = probability * 100
    else:
        predicted_class = 'bike'
        confidence = (1 - ... ) * 100

    plt.imshow(...)
    plt.title(f"Predicted: {...} ({confidence:.2f}%)")
    plt.axis('off')
    plt.show()

    return ...
```

```
image_to_classify = '...'
predicted_label = classify_new_image(...)
```



DETAILS OF CODELAB'S WORK

DETAILS	POINT
Load Dataset	10
Preprocessing Data	10
Build CNN Model	20
Compile Model	20
Training Model	10
Evaluate Model	10
Image Classification Function	20
TOTAL	100



LAB ASSIGNMENT (90%)

Dataset : [\[ASSIGNMENT DATASET\]](#)

Your task is to build a CNN model to classify images of horses vs. donkeys into one of these two classes.

1. **Dataset Exploration (5%)**
 - Load the given dataset, then check the dataset size (10 points)
 - Visualize 5 example images from each class (90 points)
2. **Data Preprocessing (5%)**
 - Normalize image pixel values (30 points)
 - Perform preprocessing as needed (DO NOT PERFORM DATA AUGMENTATION) (70 points)
3. **Building a CNN Model (10%)**
 - Create a simple CNN model (Convolution, Pooling, Fully Connected, and output layer with sigmoid activation). (100 points)
4. **Model Training (10%)**
 - Use a loss function and an optimizer (100 points)
5. **Model Evaluation (10%)**
 - Test the model using test data (70 points)
 - Calculate metrics such as accuracy, precision, recall, and F1-score (30 points)
6. **Visualization of Results (5%)**
 - Display loss and accuracy plots for training and validation data (30 points)
 - Display 5 images from the test data along with the model's predictions and original labels (35 points)
 - Create a model function to display prediction results. Provide live image input to the assistant. (35 Points)

Explain the overall model performance to the assistant, and provide suggestions to improve model performance. **You may not use (copy-paste) code from the material or codelab.**



FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment	Dataset Exploration	5%
	Data Preprocessing	5%
	Build CNN Model	10%
	Training Model	10%
	Evaluate Model	10%
	Result Visualization and Function Output	5%
Explanation to Assistant		45%
Total		100%

